

Table of Contents

1.	Case Study Selection and 3NF/ER Modeling	3
1.1	Chosen OLTP System & OLTP Business Processes Involved	3
1.2	Entity-Relationship Diagram (ERD)	4
1.3	Conceptual Schema for the Data Warehouse	6
2.	Dimensional Model Design	7
2.1	Business Process Declaration	7
2.2	Grain Declaration & Level of Detail	7
2.3	Dimension Identification	8
2.4	Building a Date Dimension with Malaysian Public Holidays	8
2.5	Fact Definition	12
3.	ETL Process Implementation	13
3.1	Extract Phase	13
3.2	Transform & Load Phase	16
4.	SQL Implementation	19
4.1	Stored Procedure	19
4.2	Trigger for automation	20
4.3	User-Defined Function	22
4.4	Complex Query	23
4.5	Group By and Advanced Grouping	25
4.6	View	26
4.7	One Advanced SQL Features Not Covered in Class (Window Functions)	26
4.8	Compare row vs. column storage vs Memory-Optimized Tables (MOT)	27
5.	Performance Optimization	30
5.1	Index critical columns (e.g., foreign keys, frequently queried fields).	30
5.2	Implement partitioning to improve query performance.	32
5.	Conclusion	34
6.	Appendix A: Instructions on Importing Missing Records into Database	35

1. Case Study Selection and 3NF/ER Modeling

1.1 Chosen OLTP System & OLTP Business Processes Involved

We selected the Sakila database as our case study because it is specifically designed to manage data for a DVD rental business. It efficiently supports daily operations by handling large volumes of quick online transactions with ease.

The list below is some of the OLTP Business Processes Involved in the System:

1. **Orders and Transactions:**

The payment and rental tables manage customer rental transactions, including payment processing and tracking which movies have been rented and returned.

2. **Inventory Management:**

The inventory table tracks the availability of DVDs across different stores. This ensures efficient stock management and minimizes rental conflicts.

3. **Customer Management:**

The customer table stores details about customers, enabling personalized service and record-keeping for rentals and payments.

4. **Store and Staff Operations:**

The store and staff tables manage employee and store-related details, ensuring smooth daily operations in different locations.

5. **Film and Category Management:**

The film, film_actor, film_category, and language tables organize information about available movies, their categories, actors, and languages, allowing customers to browse and choose DVDs.

6. Location Management:

The address, city, and country tables manage hierarchical geographical data, supporting both store and customer locations.

7. Staff Performance:

The staff table, in conjunction with the rental and payment tables, tracks the productivity of employees in facilitating customer transactions. It records metrics such as the number of rentals processed, unique customers served, and revenue generated by each staff member. This information enables the management to evaluate staff contributions and improve operational efficiency.

1.2 Entity-Relationship Diagram (ERD)

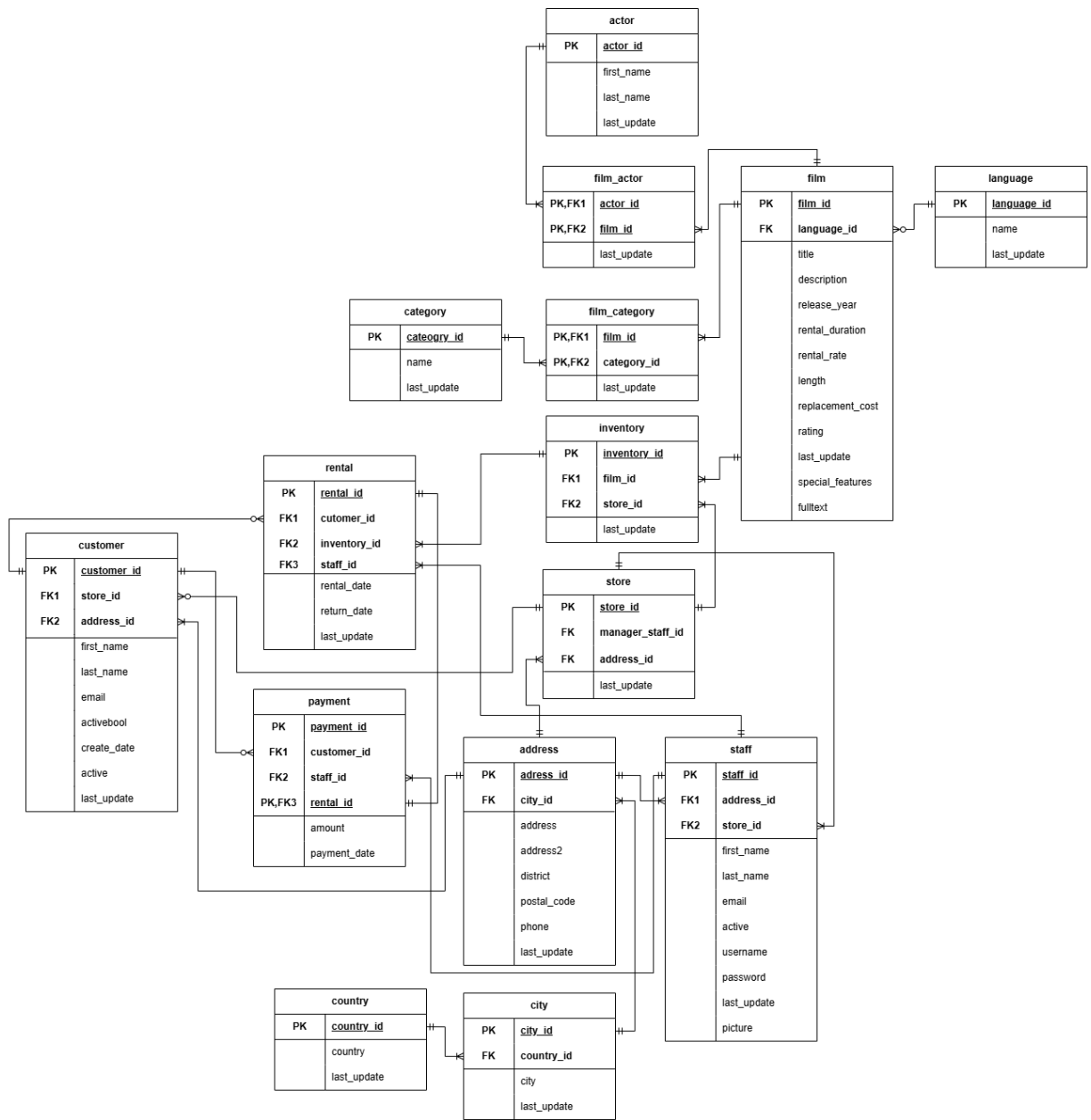


Figure 1 Entity Relationship Diagram of Sakila Database

1.3 Conceptual Schema for the Data Warehouse

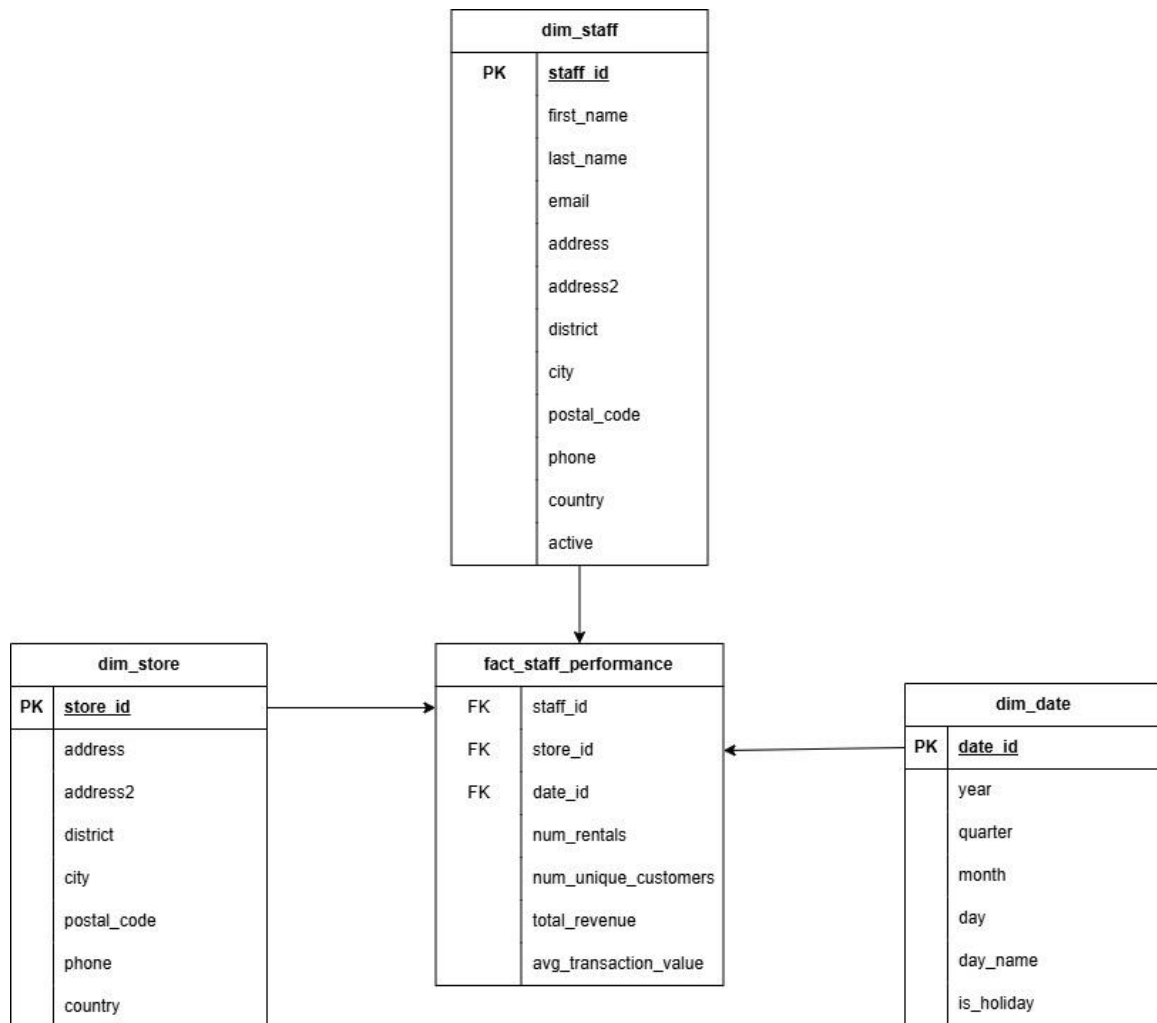


Figure 2 Star Schema for Staff Performance

2. Dimensional Model Design

2.1 Business Process Declaration

The chosen business process for the dimensional model is the evaluation of **staff performance** in terms of customer interactions and revenue generation. This process involves analyzing metrics such as the number of rentals, unique customers served, total revenue generated, and the average transaction value. By analyzing staff performance metrics, management can identify areas where employees need training, recognize high performers, and allocate resources more effectively. This helps improve employee efficiency, reduce turnover, and enhance service quality. It also ensures that the workforce is aligned with the company's goals, supporting growth and maintaining a competitive edge.

2.2 Grain Declaration & Level of Detail

The grain of the fact_staff_performance table is:

*“Each **row** represents the **performance** of a **specific staff member** in a **specific store** on a **specific date**. ”*

This definition specifies the level of detail for each row, tying together staff, store, and date dimensions.

The level of detail (granularity) for the fact_staff_performance table is:

“Daily staff performance metrics at the individual staff-store level.”

This granularity ensures the fact table tracks performance metrics for each staff member at a specific store on a daily basis.

2.3 Dimension Identification

Table 1: Dimension Table Names and Their Attributes

No.	Dimension Name	Definition	Attributes
1.	dim_staff	Describes staff members and their details, including personal information and their active status.	staff_id, first_name, last_name, email, address, address2, district, city, postal_code, phone, country, active
2.	dim_store	Represents the physical store locations and their contact details.	store_id, address, address2, district, city, postal_code, phone, country
3.	dim_date	Provides temporal context for facts by representing calendar dates and related attributes.	date_id, year, quarter, month, day, day_name, is_holiday

2.4 Building a Date Dimension with Malaysian Public Holidays

Step 1: Determine the Minimum and Maximum Dates

The date_id for the date dimension is derived from the rental_date attribute in the rental table. To identify the relevant years for extracting Malaysian public holidays, it is essential to determine the minimum and maximum dates from the rental_date field.

Input:

```
SELECT MIN(rental_date), MAX(rental_date) FROM rental;
```

Output:

```

      min      |      max
-----+-----
2010-01-01 04:21:21 | 2020-12-30 00:23:16
(1 row)

```

Figure 3 Minimum and Maximum Date Range

As shown in Figure 3, the records span from 2010 to 2020, so we need to extract the public holidays within that range.

Step 2: Extract Malaysian Public Holidays from 2010 to 2020

The [Date and Time](#) website was used to gather the Malaysian public holidays for the years 2010 to 2020. After collecting the relevant data, it was then compiled into a file named holiday_list.csv. This file will contain two columns: holiday_date and holiday_name. This CSV file will be used for marking public holidays in the date dimension table.

	A	B
1	holiday_date	holiday_name
2	1-Jan-10	New Year's Day
3	30-Jan-10	Thaipusam
4	14-Feb-10	Chinese New Year's Day
5	15-Feb-10	Second Day of Chinese New Year
6	15-Feb-10	Chinese New Year Holiday

Figure 4 Sample List of Records from "holiday_list.csv" file

Step 3: Create the holiday_list Table into the Database

Input:


```
CREATE TABLE holiday_list (
    holiday_date DATE NOT NULL, -- Date of the public holiday
    holiday_name VARCHAR(100) NOT NULL -- Name of the public holiday
);
```

Step 4: Copy Data into the holiday_list Table

Input:

```
COPY holiday_list FROM '/home/omm/holiday_list.csv' DELIMITER ',' CSV
HEADER;
```

Step 5: Create Date Dimension Table called “dim_date”

The query creates and populates the dim_date table with date-related data, including information about whether each date is a holiday. Each row represents a unique date, along with its associated year, quarter, month, day, day name, and holiday status.

Input:

```
CREATE TABLE dim_date (
    date_id DATE PRIMARY KEY, -- Unique identifier for the date
    year INT NOT NULL, -- Year (e.g., 2024)
    quarter INT NOT NULL, -- Quarter of the year (1-4)
    month INT NOT NULL, -- Month number (1-12)
    day INT NOT NULL, -- Day of the month (1-31)
    day_name VARCHAR(20) NOT NULL, -- Day name (e.g., 'Monday')
    is_holiday BOOLEAN DEFAULT FALSE -- Indicates if the date is a holiday
    (default: false)
);
```

Step 6: Create Date Dimension Table called “dim_date”

The query populates the dim_date table with date-related details (year, quarter, month, day, day name) from the rental table, while also marking each date as a holiday or not based on the holiday_list table.

Input:

```
INSERT INTO dim_date (date_id, year, quarter, month, day, day_name, is_holiday)
SELECT
```

```

DISTINCT DATE_TRUNC('day', r.rental_date) AS date_id, -- Truncate timestamp to 00:00:00 and remove
duplicates
EXTRACT(YEAR FROM DATE_TRUNC('day', r.rental_date)) AS year, -- Extract year from truncated date
EXTRACT(QUARTER FROM DATE_TRUNC('day', r.rental_date)) AS quarter, -- Extract quarter from
truncated date
EXTRACT(MONTH FROM DATE_TRUNC('day', r.rental_date)) AS month, -- Extract month from truncated date
EXTRACT(DAY FROM DATE_TRUNC('day', r.rental_date)) AS day, -- Extract day from truncated date
TO_CHAR(DATE_TRUNC('day', r.rental_date), 'Day') AS day_name, -- Get the day name
CASE
    WHEN EXISTS (
        SELECT 1
        FROM holiday_list
        WHERE holiday_date = DATE_TRUNC('day', r.rental_date) -- Compare only the date part
    ) THEN 1
    ELSE 0
END AS is_holiday -- Determine if it's a holiday
FROM
rental r; -- Assuming rental is the table with rental data

```

The holiday part of the query checks if each rental_date exists in the holiday_list table and sets the is_holiday field to 1 (TRUE) if it's a match, or 0 (FALSE) if it's not, indicating whether the date is a public holiday.

Step 7: Test by Selecting Records with Public Holidays and Non-Holiday

The query below will return the first 3 rows from the dim_date table that corresponds to public holidays, along with the associated holiday_name from the holiday_list table.

Input:

```

SELECT date_id, year, month, day, holiday_name, is_holiday
FROM dim_date d
JOIN holiday_list h ON d.date_id = h.holiday_date
WHERE d.is_holiday = TRUE
ORDER BY d.date_id
LIMIT 3;

```

Output:

date_id	year	month	day	holiday_name	is_holiday
2010-01-01 00:00:00	2010	1	1	New Year's Day	t
2010-01-30 00:00:00	2010	1	30	Thaipusam	t
2010-02-14 00:00:00	2010	2	14	Chinese New Year's Day	t

(3 rows)

The next query below will return the first 2 rows from the dim_date table where the is_holiday field is FALSE, marking those dates as "Not a Holiday" in the holiday_name field.

Input:

```
SELECT date_id, year, month, day, 'Not a Holiday' AS holiday_name,
is_holiday
FROM dim_date d
WHERE d.is_holiday = FALSE
LIMIT 2;
```

Output:

date_id	year	month	day	holiday_name	is_holiday
2018-05-03 00:00:00	2018	5	3	Not a Holiday	f
2020-10-09 00:00:00	2020	10	9	Not a Holiday	f

(2 rows)

2.5 Fact Definition

The following are the numeric facts in the **fact_staff_performance** table:

1. **num_rentals:** The total number of rentals handled by a specific staff member at a specific store on a specific date.
2. **num_unique_customers:** The total number of unique customers served by a specific staff member at a specific store on a specific date.

3. **total_revenue:** The total revenue generated by a specific staff member at a specific store on a specific date.
4. **avg_transaction_value:** The average value of transactions handled by a specific staff member at a specific store on a specific date.

3. ETL Process Implementation

This section defines the data sources and outlines the ETL workflow, including data cleaning, transformation rules, and data quality checks. It also details the implementation of the ETL pipeline using openGauss to load data into the dimensional model.

3.1 Extract Phase

Identify source tables: store, staff, address, city, country, rental, and payment.

Table 2: The relationship between data warehousing business terms and data sources

Business Term	Data Source	Table.Column Name
Staff Performance Fact Table:		
Staff ID		fact_staff_performance.staff_id
Store ID		fact_staff_performance.store_id

Date ID		fact_staff_performance.date_id
Number of Rentals	rental.rental_id	fact_staff_performance.num_rentals
Number of Unique Customers	payment.customer_id	fact_staff_performance.num_unique_customers
Total Revenue	payment.amount	fact_staff_performance.total_revenue
Average Transaction Value	calculated:total_revenue divided by num_rentals	fact_staff_performance.avg_transaction_value
Date Dimension Table:		
Date ID	rental.rental_date	dim_date.date_id
Year	derived from rental.rental_date	dim_date.year
Quarter	derived from rental.rental_date	dim_date.quarter
Month	derived from rental.rental_date	dim_date.month
Day	derived from rental.rental_date	dim_date.day
Day Name	derived from rental.rental_date	dim_date.day_name
Holiday or Not a Holiday	derived: IF EXISTS (SELECT 1 FROM holiday_list WHERE holiday_date = rental.rental_date) THEN 1 ELSE 0	dim_date.is_holiday
Staff Dimension Table:		
Staff ID	staff.staff_id	dim_staff.staff_id
First Name	staff.first_name	dim_staff.first_name
Last Name	staff.last_name	dim_staff.last_name
Email	staff.email	dim_staff.email
Address	address.address	dim_staff.address
Address 2	address.address2	dim_staff.address2
District	address.district	dim_staff.district
City	city.city	dim_staff.city
Postal Code	address.postal_code	dim_staff.postal_code
Phone	address.phone	dim_staff.phone
Country	country.country	dim_staff.country

Active	staff.active	dim_staff.active
Store Dimension Table:		
Store ID	store.store_id	dim_store.store_id
Address	address.address	dim_store.address
Address 2	address.address2	dim_store.address2
District	address.district	dim_store.district
City	city.city	dim_store.city
Postal Code	address.postal_code	dim_store.postal_code
Phone	address.phone	dim_store.phone
Country	country.country	dim_store.country

Then create the dimension tables, followed by the fact tables.

Create dim_store by joining store, address, city, and country.

```
CREATE TABLE dim_store (
    store_id INT PRIMARY KEY,           -- Unique identifier for each store
    address VARCHAR(50) NOT NULL,       -- Address line 1
    address2 VARCHAR(50),               -- Address line 2
    district VARCHAR(20) NOT NULL,      -- District of the address
    city VARCHAR(50) NOT NULL,          -- City name
    postal_code VARCHAR(10),            -- Postal code
    phone VARCHAR(20) NOT NULL,         -- Phone number for the store
    country VARCHAR(50) NOT NULL        -- Country name
);
```

Create dim_staff by joining staff, address, city, and country with additional active status.

```
CREATE TABLE dim_staff (
    staff_id INT PRIMARY KEY,           -- Unique identifier for
    staff                                -- Staff
    first_name VARCHAR(45) NOT NULL,    -- Staff's first name
    last_name VARCHAR(45) NOT NULL,     -- Staff's last name
    email VARCHAR(50),                  -- Staff's email address
    address VARCHAR(50) NOT NULL,       -- Address line 1
    address2 VARCHAR(50),               -- Address line 2
    district VARCHAR(20) NOT NULL,      -- District of the address
    city VARCHAR(50) NOT NULL,          -- City name
    postal_code VARCHAR(10),            -- Postal code
    phone VARCHAR(20) NOT NULL,         -- Phone number for the staff
    country VARCHAR(50) NOT NULL,       -- Country name
    active BOOLEAN NOT NULL            -- Whether the staff is active
);
```

Create dim_date by extracting date components (year, month, day) and deriving day_name and is_holiday.

```
CREATE TABLE dim_date (
    date_id DATE PRIMARY KEY,          -- Unique identifier for the date
    year INT NOT NULL,                 -- Year (e.g., 2024)
    quarter INT NOT NULL,              -- Quarter of the year (1-4)
    month INT NOT NULL,                -- Month number (1-12)
    day INT NOT NULL,                  -- Day of the month (1-31)
    day_name VARCHAR(20) NOT NULL,     -- Day name (e.g., 'Monday')
    is_holiday BOOLEAN DEFAULT FALSE   -- Indicates if the date is a holiday
    (default: false)
);
```

Create fact_staff_performance with aggregated metrics like num_rentals, num_unique_customers, total_revenue, and avg_transaction_value.

Note: Column-store tables do not allow foreign key constraints to be defined according to [OpenGauss](#).

```
CREATE TABLE fact_staff_performance (
    staff_id INT NOT NULL,
    store_id INT NOT NULL,
    date_id DATE NOT NULL,
    num_rentals INT,
    num_unique_customers INT,
    total_revenue DECIMAL(10, 2),
    avg_transaction_value DECIMAL(10, 2),

    CONSTRAINT pk_fact_staff_performance PRIMARY KEY (staff_id, store_id,
    date_id)
)
WITH (orientation = 'column');
```

3.2 Transform & Load Phase

Insert transformed data into: dim_store, dim_staff, and dim_date.

```
INSERT INTO dim_store (store_id, address, address2, district, city,
postal_code, phone, country)
SELECT
    s.store_id,
    a.address,
    a.address2,
    a.district,
    c.city,
    a.postal_code,
    a.phone,
    co.country
FROM
    store s
```

```

JOIN
    address a ON s.address_id = a.address_id
JOIN
    city c ON a.city_id = c.city_id
JOIN
    country co ON c.country_id = co.country_id;

```

```

INSERT INTO dim_staff (
    staff_id,
    first_name,
    last_name,
    email,
    address,
    address2,
    district,
    city,
    postal_code,
    phone,
    country,
    active
)
SELECT
    st.staff_id,           -- Staff ID from the staff table
    st.first_name,         -- First name from the staff table
    st.last_name,          -- Last name from the staff table
    st.email,              -- Email from the staff table
    a.address,             -- Address line 1 from the address table
    a.address2,            -- Address line 2 (if available)
    a.district,            -- District from the address table
    c.city,                -- City name from the city table
    a.postal_code,         -- Postal code from the address table
    a.phone,               -- Phone number from the address table
    co.country,            -- Country name from the country table
    st.active              -- Active status from the staff table
FROM
    staff st
JOIN
    address a ON st.address_id = a.address_id
JOIN
    city c ON a.city_id = c.city_id
JOIN
    country co ON c.country_id = co.country_id;

```

```

INSERT INTO dim_date (date_id, year, quarter, month, day, day_name, is_holiday)
SELECT
    DISTINCT DATE_TRUNC('day', r.rental_date) AS date_id, -- Truncate timestamp to 00:00:00 and
    remove duplicates
    EXTRACT(YEAR FROM DATE_TRUNC('day', r.rental_date)) AS year, -- Extract year from truncated
    date
    EXTRACT(QUARTER FROM DATE_TRUNC('day', r.rental_date)) AS quarter, -- Extract quarter from
    truncated date
    EXTRACT(MONTH FROM DATE_TRUNC('day', r.rental_date)) AS month, -- Extract month from
    truncated date
    EXTRACT(DAY FROM DATE_TRUNC('day', r.rental_date)) AS day, -- Extract day from truncated
    date

```



```

        TO_CHAR(DATE_TRUNC('day', r.rental_date), 'Day') AS day_name, -- Get the day name
    CASE
        WHEN EXISTS (
            SELECT 1
            FROM holiday_list
            WHERE holiday_date = DATE_TRUNC('day', r.rental_date)
        ) THEN 1
        ELSE 0
    END AS is_holiday -- Determine if it's a holiday
FROM
    rental r;

```

Load metrics into fact_staff_performance, maintaining references to dimension tables.

```

INSERT INTO fact_staff_performance (
    staff_id,
    store_id,
    date_id,
    num_rentals,
    num_unique_customers,
    total_revenue,
    avg_transaction_value
)
SELECT
    ds.staff_id,
    dsto.store_id,
    dd.date_id,
    COUNT(r.rental_id) AS num_rentals,
    COUNT(DISTINCT p.customer_id) AS num_unique_customers,
    COALESCE(SUM(p.amount), 0) AS total_revenue,
    CASE
        WHEN COUNT(r.rental_id) = 0 THEN 0
        ELSE COALESCE(SUM(p.amount), 0) / COUNT(r.rental_id)
    END AS avg_transaction_value
FROM
    rental r
LEFT JOIN
    payment p ON r.rental_id = p.rental_id
LEFT JOIN
    inventory i ON r.inventory_id = i.inventory_id
JOIN
    dim_staff ds ON r.staff_id = ds.staff_id
JOIN
    dim_store dsto ON i.store_id = dsto.store_id
JOIN
    dim_date dd ON DATE_TRUNC('day', r.rental_date) = dd.date_id

```

```
GROUP BY
    ds.staff_id,
    dsto.store_id,
    dd.date_id
HAVING COUNT(DISTINCT p.customer_id) > 0;
```

4. SQL Implementation

4.1 Stored Procedure

Note: There are problems with using PROCEDURE, so we resorted to using FUNCTION instead.

The purpose of the following function is to insert new data into the fact_staff_performance table.

```
CREATE OR REPLACE FUNCTION insert_staff_performance(
    staff_id INT,
    store_id INT,
    rental_date DATE,
    num_rentals INT,
    num_unique_customers INT,
    total_revenue DECIMAL(10, 2),
    avg_transaction_value DECIMAL(10, 2)
)
RETURNS VOID
AS
$$
BEGIN
    INSERT INTO fact_staff_performance (
        staff_id,
        store_id,
        date_id,
        num_rentals,
        num_unique_customers,
        total_revenue,
        avg_transaction_value
    )
    VALUES (
        staff_id,
        store_id,
        rental_date,
        num_rentals,
        num_unique_customers,
        total_revenue,
        avg_transaction_value
    );

    RAISE NOTICE 'Data inserted successfully for staff_id: %, date: %', staff_id,
    rental_date;
END;
$$
```

```
LANGUAGE plpgsql;
```

OUTPUT:

```
dvd=# select * from fact_staff_performance WHERE staff_id = 1 AND store_id=1 AND date_id='2020-12-21';
 staff_id | store_id | date_id | num_rentals | num_unique_customers | total_revenue | avg_transaction_value 
-----+-----+-----+-----+-----+-----+-----
(0 rows)

dvd=# SELECT insert_staff_performance(1, 1, '2020-12-21', 10, 5, 100.00, 10.00);
NOTICE: Data inserted successfully for staff_id: 1, date: 2020-12-21 00:00:00
CONTEXT: referenced column: insert_staff_performance
 insert_staff_performance
-----
(1 row)

dvd=# select * from fact_staff_performance WHERE staff_id = 1 AND store_id=1 AND date_id='2020-12-21';
 staff_id | store_id | date_id | num_rentals | num_unique_customers | total_revenue | avg_transaction_value 
-----+-----+-----+-----+-----+-----+-----
      1  |      1  | 2020-12-21 00:00:00 |      10  |           5  |      100.00  |          10.00 
(1 row)
```

4.2 Trigger for automation

The purpose of this trigger is to log INSERT/UPDATE changes on the dim_staff table, into the audit_log_dim_staff table.

Step 1: Create the Audit Log Table

```
CREATE TABLE audit_log_dim_staff (
    audit_id SERIAL PRIMARY KEY,           -- Unique identifier for each log
    staff_id INT,                          -- ID of the staff affected
    action VARCHAR(10),                    -- Type of action
    first_name VARCHAR(45),                -- Old or new first name
    last_name VARCHAR(45),                 -- Old or new last name
    email VARCHAR(50),                     -- Old or new email
    address VARCHAR(50),                   -- Old or new address
    address2 VARCHAR(50),                   -- Old or new address2
    district VARCHAR(20),                   -- Old or new district
    city VARCHAR(50),                       -- Old or new city
    postal_code VARCHAR(10),                -- Old or new postal_code
    phone VARCHAR(20),                      -- Old or new phone
    country VARCHAR(50),                    -- Old or new country
    action_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP -- Timestamp of
the change
);
```

Step 2: Create the Trigger Function

```
CREATE OR REPLACE FUNCTION log_dim_staff_changes()
RETURNS TRIGGER AS $$
BEGIN
    -- Log INSERT action
    IF TG_OP = 'INSERT' THEN
        INSERT INTO audit_log_dim_staff (staff_id, action, first_name,
last_name, email, address, address2, district, city, postal_code, phone,
country)
```

```

VALUES (NEW.staff_id, 'INSERT', NEW.first_name, NEW.last_name,
NEW.email, NEW.address, NEW.address2, NEW.district, NEW.city,
NEW.postal_code, NEW.phone, NEW.country);
-- Log UPDATE action
ELSIF TG_OP = 'UPDATE' THEN
INSERT INTO audit_log_dim_staff (staff_id, action, first_name,
last_name, email, address, address2, district, city, postal_code, phone,
country)
VALUES (NEW.staff_id, 'UPDATE', NEW.first_name, NEW.last_name,
NEW.email, NEW.address, NEW.address2, NEW.district, NEW.city,
NEW.postal_code, NEW.phone, NEW.country);
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

Step 3: Create the Trigger

```

CREATE TRIGGER trigger_log_dim_staff_changes
AFTER INSERT OR UPDATE ON dim_staff
FOR EACH ROW
EXECUTE PROCEDURE log_dim_staff_changes();

```

Testing INSERT & UPDATE statements BEFORE Trigger:

```

-- Insert a new staff member
INSERT INTO dim_staff (staff_id, first_name, last_name, email, address,
address2, district, city, postal_code, phone, country, active)
VALUES (101, 'John', 'Doe', 'john.doe@example.com', '123 Main St', 'Apt 4B',
'District 1', 'City A', '12345', '123-456-7890', 'Country X', true);
INSERT 0 1

-- Update the staff member's first name and email
UPDATE dim_staff
SET first_name = 'Jane', email = 'jane.doe@example.com'
WHERE staff_id = 101;

```

Output before Trigger:

```

dvd=# -- Insert a new staff member
dvd=# INSERT INTO dim_staff (staff_id, first_name, last_name, email, address, address2, district, city, postal_code, phone, country, active)
dvd=# VALUES (101, 'John', 'Doe', 'john.doe@example.com', '123 Main St', 'Apt 4B', 'District 1', 'City A', '12345', '123-456-7890', 'Country X', true);
INSERT 0 1
dvd=# -- Update the staff member's first name and email
dvd=# UPDATE dim_staff
dvd=# SET first_name = 'Jane', email = 'jane.doe@example.com'
dvd=# WHERE staff_id = 101;
UPDATE 1
dvd=# SELECT * FROM audit_log_dim_staff;
 audit_id | staff_id | action | first_name | last_name | email | address | address2 | district | city | postal_code | phone | country | action_timestamp
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
(0 rows)

```

Testing INSERT & UPDATE Statements BEFORE trigger:

```
-- Insert a new staff member
INSERT INTO dim_staff (staff_id, first_name, last_name, email,
address, address2, district, city, postal_code, phone, country,
active)
VALUES (102, 'Michael', 'Smith', 'michael.smith@example.com',
'456 Oak St', 'Apt 12C', 'District 2', 'City B', '67890', '987-
654-3210', 'Country Y', true);

-- Update the staff member's first name and email
UPDATE dim_staff
SET first_name = 'Mike', email = 'mike.smith@example.com'
WHERE staff_id = 102;
```

Output After Trigger:

```
dvd=# CREATE TRIGGER trigger_log_dim_staff_changes
dvd=# AFTER INSERT OR UPDATE ON dim_staff
dvd=# FOR EACH ROW
dvd=# EXECUTE PROCEDURE log_dim_staff_changes();
CREATE TRIGGER
dvd=# -- Insert a new staff member
dvd=# INSERT INTO dim_staff (staff_id, first_name, last_name, email, address, address2, district, city, postal_code, phone, country, active)
dvd=# VALUES (102, 'Michael', 'Smith', 'michael.smith@example.com', '456 Oak St', 'Apt 12C', 'District 2', 'City B', '67890', '987-654-3210', 'Country Y', true);
INSERT 0 1
dvd=# -- Update the staff member's first name and email
dvd=# UPDATE dim_staff
dvd=# SET first_name = 'Mike', email = 'mike.smith@example.com'
dvd=# WHERE staff_id = 102;
UPDATE 1
dvd=# SELECT * FROM audit_log_dim_staff;
 audit_id | staff_id | action | first_name | last_name | email | address | address2 | district | city | postal_code | phone | country | action_timestamp
-----
1 | 102 | INSERT | Michael | Smith | michael.smith@example.com | 456 Oak St | Apt 12C | District 2 | City B | 67890 | 987-654-3210 | Country Y | 2024-12-22 16:59:39.299675
2 | 102 | UPDATE | Mike | Smith | mike.smith@example.com | 456 Oak St | Apt 12C | District 2 | City B | 67890 | 987-654-3210 | Country Y | 2024-12-22 16:59:45.747061
(2 rows)
```

4.3 User-Defined Function

The purpose of this function is to calculate the average transaction a staff makes in a given date range.

```
CREATE OR REPLACE FUNCTION calculate_avg_transaction_value(
    p_staff_id INT,          -- Staff ID for which the average will be calculated
    p_start_date DATE,       -- Start date of the period
    p_end_date DATE         -- End date of the period
)
RETURNS DECIMAL(10, 2) AS $$
DECLARE
    total_revenue_daterange DECIMAL(10, 2);
    total_rentals_daterange INT;
    avg_transaction_daterange DECIMAL(10, 2);
BEGIN
    -- Calculate total revenue and total rentals for the specified staff and date range
    SELECT
        COALESCE(SUM(total_revenue), 0),
        COALESCE(SUM(num_rentals), 0)
    INTO total_revenue_daterange, total_rentals_daterange
    FROM fact_staff_performance
```

```

WHERE staff_id = p_staff_id
AND date_id BETWEEN p_start_date AND p_end_date;

-- Calculate average transaction value
IF total_rentals_daterange > 0 THEN
    avg_transaction_daterange := total_revenue_daterange / total_rentals_daterange;
ELSE
    avg_transaction_daterange := 0;
END IF;

-- Return the calculated average transaction value
RETURN avg_transaction_daterange;
END;
$$ LANGUAGE plpgsql;

```

OUTPUT:

```

dvd=# SELECT calculate_avg_transaction_value(1, '2010-01-01', '2010-12-31');
calculate_avg_transaction_value
-----
                    52.57
(1 row)

```

4.4 Complex Query

The purpose of this query is to rank staff members based on their workload (number of rentals) during the holidays in 2020, categorize them into high, medium, or low workload groups, and return the top 3 staff members from each category.

```

WITH RankedWorkload AS (
    SELECT
        ds.staff_id,
        ds.first_name,
        ds.last_name,
        dsto.store_id,
        dd.year,
        dd.is_holiday,
        COUNT(r.rental_id) AS num_rentals, -- Workload defined by number of
rentals
        COUNT(DISTINCT p.customer_id) AS num_unique_customers,
        CASE
            WHEN COUNT(r.rental_id) >= 6 THEN 'High Workload' -- High workload if
rentals >= 6
            WHEN COUNT(r.rental_id) >= 4 THEN 'Medium Workload' -- Medium workload
if rentals between 4 and 5
            ELSE 'Low Workload' -- Low workload for rentals < 4
        END AS workload_category,
        ROW_NUMBER() OVER (
            PARTITION BY
                CASE

```

```

        WHEN COUNT(r.rental_id) >= 6 THEN 'High Workload'
        WHEN COUNT(r.rental_id) >= 4 THEN 'Medium Workload'
        ELSE 'Low Workload'
    END
    ORDER BY COUNT(r.rental_id) DESC
) AS rank
FROM
    rental r
LEFT JOIN
    payment p ON r.rental_id = p.rental_id
LEFT JOIN
    inventory i ON r.inventory_id = i.inventory_id
JOIN
    dim_staff ds ON r.staff_id = ds.staff_id
JOIN
    dim_store dsto ON i.store_id = dsto.store_id
JOIN
    dim_date dd ON DATE_TRUNC('day', r.rental_date) = dd.date_id
WHERE
    dd.year = 2020
    AND dd.is_holiday = TRUE
GROUP BY
    ds.staff_id, ds.first_name, ds.last_name, dsto.store_id, dd.year,
dd.is_holiday
)
SELECT
    staff_id,
    first_name,
    last_name,
    store_id,
    year,
    num_rentals,
    workload_category
FROM
    RankedWorkload
WHERE
    rank <= 3; -- Limit to the top 3 staff per workload category

```

OUTPUT:

staff_id	first_name	last_name	store_id	year	num_rentals	workload_category
124	Jasper	Nolan	2	2020	8	High Workload
75	Tomas	Braun	2	2020	7	High Workload
14	Pauline	Rau	1	2020	7	High Workload
1	Mike	Hillyer	1	2020	3	Low Workload
2	Jon	Stephens	1	2020	3	Low Workload
4	Lois	Schroeder	1	2020	3	Low Workload
114	Trey	Collier	1	2020	5	Medium Workload
121	Noemi	Thiel	2	2020	5	Medium Workload
71	Emmanuelle	Rowe	1	2020	5	Medium Workload

(9 rows)

4.5 Group By and Advanced Grouping

The purpose of this query is to find the staff that have generated over 32000.00 in total_revenue across all the stores they work at.

```
WITH FilteredStores AS (  
  -- Step 1: Identify staff IDs with 'All Stores' rows exceeding total_revenue > 32000  
  SELECT  
    staff_id  
  FROM  
    fact_staff_performance  
  GROUP BY  
    ROLLUP(staff_id, store_id)  
  HAVING  
    store_id IS NULL AND SUM(total_revenue) > 32000  
)  
-- Step 2: Retrieve detailed rows and 'All Stores' rows for the identified staff IDs  
SELECT  
  COALESCE(CAST(staff_id AS TEXT), 'All Staff') AS staff_id,  
  COALESCE(CAST(store_id AS TEXT), 'All Stores') AS store_id,  
  COUNT(*) AS num_rentals,  
  SUM(num_unique_customers) AS total_unique_customers,  
  SUM(total_revenue) AS total_revenue,  
  AVG(avg_transaction_value) AS avg_transaction_value  
FROM  
  fact_staff_performance  
WHERE  
  staff_id IN (SELECT staff_id FROM FilteredStores)  
GROUP BY  
  CUBE(staff_id, store_id)  
ORDER BY  
  staff_id, store_id;
```

OUTPUT:

staff_id	store_id	num_rentals	total_unique_customers	total_revenue	avg_transaction_value
1	1	526	593	29097.90	48.5090114068441065
1	2	549	623	31265.30	49.5228051001821494
1	All Stores	1075	1216	60363.20	49.0267534883720930
124	1	298	320	16186.80	50.1865771812080537
124	2	327	348	17027.80	48.3945871559633028
124	All Stores	625	668	33214.60	49.2490080000000000
73	1	311	328	15794.20	47.9828295819935691
73	2	299	326	16326.30	50.0650501672240803
73	All Stores	610	654	32120.50	49.0034590163934426
All Staff	1	1135	1241	61078.90	48.8052863436123348
All Staff	2	1175	1297	64619.40	49.3468085106382979
All Staff	All Stores	2310	2538	125698.30	49.0807359307359307

4.6 View

The purpose of this view is to aggregate and present sales performance data by region (country), including total revenue, rentals, staff count, and average transaction value, sorted by total revenue.

```
CREATE VIEW sales_by_region AS
SELECT
    st.country AS region,                -- Region (country) for the sales
    SUM(fsp.total_revenue) AS total_revenue, -- Total revenue for the region
    SUM(fsp.num_rentals) AS total_rentals,   -- Total rentals for the region
    COUNT(DISTINCT fsp.staff_id) AS num_staff, -- Number of distinct staff in the region
    AVG(fsp.avg_transaction_value) AS avg_trans_value -- Average transaction value across
the region
FROM
    fact_staff_performance fsp          -- Fact table for staff performance
JOIN
    dim_store st ON fsp.store_id = st.store_id -- Join with dim_store to get region (country)
GROUP BY
    st.country                          -- Group by country (region)
ORDER BY
    total_revenue DESC;                -- Sort by total revenue for easy analysis
```

Output:

```
dvd=# SELECT * FROM sales_by_region;
 region | total_revenue | total_rentals | num_staff | avg_trans_value
-----+-----+-----+-----+-----
Australia |      830488.20 |         16889 |         55 | 49.4381285130301482
Canada   |      818921.50 |         16807 |         55 | 48.9951141025641026
(2 rows)
```

4.7 One Advanced SQL Features Not Covered in Class (Window Functions)

The purpose of this query is a top 10 ranking of staff in store 2, based on the total_revenue they made on days which are holidays.

```
WITH RankedStaff AS (
SELECT
    ds.first_name,
    ds.last_name,
    fsp.store_id,
    dd.date_id,
    fsp.num_rentals,
    fsp.total_revenue,
    RANK() OVER (
        PARTITION BY fsp.store_id
```

```

        ORDER BY SUM(fsp.total_revenue) DESC
    ) AS revenue_rank_within_store,
    dd.is_holiday
FROM
    fact_staff_performance fsp
JOIN
    dim_staff ds ON fsp.staff_id = ds.staff_id
JOIN
    dim_date dd ON fsp.date_id = dd.date_id
WHERE fsp.store_id = 2 AND dd.is_holiday = TRUE
GROUP BY
    ds.first_name, ds.last_name, fsp.store_id, dd.date_id, fsp.num_rentals,
    fsp.total_revenue, fsp.store_id, fsp.staff_id
)

SELECT
    first_name,
    last_name,
    store_id,
    date_id,
    num_rentals,
    total_revenue,
    revenue_rank_within_store,
    is_holiday
FROM RankedStaff
WHERE revenue_rank_within_store <=10;

```

OUTPUT:

first_name	last_name	store_id	date_id	num_rentals	total_revenue	revenue_rank_within_store	is_holiday
Jon	Stephens	2	2020-01-27 00:00:00	3	248.30	1	t
Uriel	Kohler	2	2016-06-04 00:00:00	3	207.20	2	t
Jasper	Nolan	2	2020-07-31 00:00:00	2	173.20	3	t
Nat	Zboncak	2	2017-01-29 00:00:00	4	172.70	4	t
Ariane	Bosco	2	2020-07-31 00:00:00	2	155.70	5	t
Leslie	Orn	2	2016-12-31 00:00:00	2	154.20	6	t
Blaise	Schmitt	2	2020-08-02 00:00:00	2	150.70	7	t
Trey	Collier	2	2020-01-26 00:00:00	3	149.30	8	t
Jasper	Nolan	2	2020-05-24 00:00:00	2	146.60	9	t
Tomas	Braun	2	2010-12-31 00:00:00	2	142.90	10	t

4.8 Compare row vs. column storage vs Memory-Optimized Tables (MOT)

Step 1: Create the Row-Based Table

```

CREATE TABLE fact_staff_performance_row (
    staff_id INT,
    store_id INT,
    date_id DATE,
    num_rentals INT,
    num_unique_customers INT,
    total_revenue DECIMAL(10, 2),
    avg_transaction_value DECIMAL(10, 2)
) WITH (orientation=row);

```

Step 2: Create the Memory-Optimized Table

We first need to turn off `enable_incremental_checkpoints` to create an MOT:

```
gs_guc set -N all -I all -c
"enable_incremental_checkpoint='off'"
```

Then restart the server:

```
gs_om -t stop && gs_om -t start
```

```
CREATE FOREIGN TABLE fact_staff_performance_mot (
    staff_id INT,
    store_id INT,
    date_id DATE,
    num_rentals INT,
    num_unique_customers INT,
    total_revenue DECIMAL(10, 2),
    avg_transaction_value DECIMAL(10, 2)
);
```

Step 3: Insert the new tables with existing dataset

```
INSERT INTO fact_staff_performance_row
SELECT * FROM fact_staff_performance;
```

```
INSERT INTO fact_staff_performance_mot
SELECT * FROM fact_staff_performance;
```

Step 4: Use EXPLAIN ANALYZE to measure the performance of a query

```
EXPLAIN ANALYZE
SELECT staff_id, SUM(num_rentals)
FROM fact_staff_performance
GROUP BY staff_id;
```

```
EXPLAIN ANALYZE
SELECT staff_id, SUM(num_rentals)
FROM fact_staff_performance_row
GROUP BY staff_id;
```

```
EXPLAIN ANALYZE
SELECT staff_id, SUM(num_rentals)
FROM fact_staff_performance_mot
GROUP BY staff_id;
```

Output:

```
dvd=# EXPLAIN ANALYZE
dvd=#
dvd=# SELECT staff_id, SUM(num_rentals)
dvd=#
dvd=# FROM fact_staff_performance
dvd=#
dvd=# GROUP BY staff_id;

                                QUERY PLAN
-----
Row Adapter  (cost=524.09..524.09 rows=55 width=16) (actual time=1.361..1.365 rows=55 loops=1)
  -> Vector Sonic Hash Aggregate  (cost=523.54..524.09 rows=55 width=16) (actual time=1.354..1.355 rows=55 loops=1)
        Group By Key: staff_id
        -> CStore Scan on fact_staff_performance  (cost=0.00..367.26 rows=31257 width=8) (actual time=0.037..0.270 rows=31257 loops=1)
Total runtime: 1.717 ms
(5 rows)

dvd=# EXPLAIN ANALYZE
dvd=#
dvd=# SELECT staff_id, SUM(num_rentals)
dvd=#
dvd=# FROM fact_staff_performance_row
dvd=#
dvd=# GROUP BY staff_id;

                                QUERY PLAN
-----
HashAggregate  (cost=761.06..762.40 rows=55 width=16) (actual time=10.564..10.568 rows=55 loops=1)
  Group By Key: staff_id
  -> Seq Scan on fact_staff_performance_row  (cost=0.00..605.57 rows=31257 width=8) (actual time=0.014..5.188 rows=31257 loops=1)
Total runtime: 10.649 ms
(4 rows)

dvd=# EXPLAIN ANALYZE
dvd=#
dvd=# SELECT staff_id, SUM(num_rentals)
dvd=#
dvd=# FROM fact_staff_performance_mot
dvd=#
dvd=# GROUP BY staff_id;

                                QUERY PLAN
-----
HashAggregate  (cost=3281.99..3283.99 rows=200 width=16) (actual time=9.946..9.954 rows=55 loops=1)
  Group By Key: staff_id
  -> Foreign Scan on fact_staff_performance_mot  (cost=0.10..3125.70 rows=31257 width=8) (actual time=0.015..4.174 rows=31257 loops=1)
        -> Memory Engine returned rows: 31257
Total runtime: 10.042 ms
(5 rows)
```

Conclusion

In conclusion, for a fact table, column stores offer the best performance for analytical queries due to their ability to efficiently read only the required columns, leverage vectorized processing, and minimize I/O with compression. Row stores perform the worst in such scenarios because they read entire rows, including irrelevant columns, leading to higher I/O and slower aggregation. Memory-optimized tables (MOTs) perform better than row stores by eliminating disk I/O through in-memory processing but are still constrained by row-oriented storage, making them less efficient for aggregation-heavy queries. Therefore, for fact tables in data warehouses, column stores are the optimal choice, while row stores and MOTs are better suited for transactional workloads.

Table 3: Query Execution Time Comparison Across Different Storage Types

Storage Type	Query Time (ms)
Columnar (fact_staff_performance)	1.717
Row-based (fact_staff_performance_row)	10.649
Memory-Optimized (fact_staff_performance_mot)	10.042

5. Performance Optimization

5.1 Index critical columns (e.g., foreign keys, frequently queried fields).

```
CREATE INDEX idx_staff_id ON fact_staff_performance (staff_id);
CREATE INDEX idx_store_id ON fact_staff_performance (store_id);
CREATE INDEX idx_date_id ON fact_staff_performance (date_id);
CREATE INDEX idx_total_revenue ON fact_staff_performance (total_revenue);
CREATE INDEX idx_first_name ON dim_staff (first_name);
CREATE INDEX idx_last_name_staff ON dim_staff (last_name);
CREATE INDEX idx_is_holiday ON dim_date(is_holiday);
```

```
Query Performance For the Given Query:
EXPLAIN ANALYZE
WITH RankedStaff AS (
```

```

SELECT
    ds.first_name,
    ds.last_name,
    fsp.store_id,
    dd.date_id,
    fsp.num_rentals,
    fsp.total_revenue,
    RANK() OVER (
        PARTITION BY fsp.store_id
        ORDER BY SUM(fsp.total_revenue) DESC
    ) AS revenue_rank_within_store,
    dd.is_holiday
FROM
    fact_staff_performance fsp
JOIN
    dim_staff ds ON fsp.staff_id = ds.staff_id
JOIN
    dim_date dd ON fsp.date_id = dd.date_id
WHERE fsp.store_id = 2 AND dd.is_holiday = TRUE
GROUP BY
    ds.first_name, ds.last_name, fsp.store_id, dd.date_id, fsp.num_rentals,
    fsp.total_revenue, fsp.store_id, fsp.staff_id
)

SELECT
    first_name,
    last_name,
    store_id,
    date_id,
    num_rentals,
    total_revenue,
    revenue_rank_within_store,
    is_holiday
FROM RankedStaff
WHERE revenue_rank_within_store <=10;

```

Before Indexing Output:

```

----- QUERY PLAN -----
--
Subquery Scan on rankedstaff (cost=660.84..684.00 rows=257 width=49) (actual time=12.562..13.426 rows=10 loops=1)
  Filter: (rankedstaff.revenue_rank_within_store <= 10)
  Rows Removed by Filter: 907
  -> WindowAgg (cost=660.84..674.35 rows=772 width=77) (actual time=12.552..13.299 rows=917 loops=1)
    -> Sort (cost=660.84..662.77 rows=772 width=77) (actual time=12.544..12.609 rows=917 loops=1)
      Sort Key: (sum(fsp.total_revenue)) DESC
      Sort Method: quicksort Memory: 168kB
    -> HashAggregate (cost=616.10..623.82 rows=772 width=77) (actual time=11.434..11.741 rows=917 loops=1)
      Group By Key: ds.first_name, ds.last_name, fsp.store_id, dd.date_id, fsp.num_rentals, fsp.total_revenue, fsp.staff_id
      -> Hash Join (cost=78.83..600.66 rows=772 width=45) (actual time=7.931..10.773 rows=917 loops=1)
        Hash Cond: (fsp.staff_id = ds.staff_id)
        -> Hash Join (cost=76.59..587.80 rows=772 width=32) (actual time=3.201..5.757 rows=917 loops=1)
          Hash Cond: (fsp.date_id = dd.date_id)
          -> Row Adapter (cost=445.40..445.40 rows=15648 width=31) (actual time=0.083..1.018 rows=15656 loops=1)
            -> CStore Scan on fact_staff_performance fsp (cost=0.00..445.40 rows=15648 width=31) (actual time=0.071..0.471 rows=15656 loops=1)
          -> Seq Scan on dim_date dd (cost=0.00..74.17 rows=194 width=9) (actual time=1.307..2.821 rows=194 loops=1)
            Filter: is_holiday
            Rows Removed by Filter: 3823
        -> Hash (cost=74.17..74.17 rows=194 width=9) (actual time=2.886..2.886 rows=194 loops=1)
          Buckets: 32768 Batches: 1 Memory Usage: 264kB
          -> Seq Scan on dim_staff ds (cost=0.00..1.55 rows=55 width=17) (actual time=1.095..1.115 rows=55 loops=1)
            Filter: (store_id = 2)
            Rows Removed by Filter: 15601
      -> Seq Scan on fact_staff_performance fsp (cost=0.00..445.40 rows=15648 width=31) (actual time=0.071..0.471 rows=15656 loops=1)
        Filter: (store_id = 2)
        Rows Removed by Filter: 15601
      Buckets: 32768 Batches: 1 Memory Usage: 259kB
    -> Seq Scan on dim_staff ds (cost=0.00..1.55 rows=55 width=17) (actual time=1.095..1.115 rows=55 loops=1)
      Filter: (store_id = 2)
      Rows Removed by Filter: 15601
Total runtime: 35.184 ms
(26 rows)

```

After Indexing Output:

```

QUERY PLAN
-----
Subquery Scan on rankedstaff (cost=620.97..644.13 rows=257 width=49) (actual time=4.971..5.679 rows=10 loops=1)
  Filter: (rankedstaff.revenue_rank_within_store <= 10)
  Rows Removed by Filter: 907
  -> WindowAgg (cost=620.97..634.48 rows=772 width=77) (actual time=4.962..5.574 rows=917 loops=1)
    -> Sort (cost=620.97..622.90 rows=772 width=77) (actual time=4.953..4.992 rows=917 loops=1)
      Sort Key: (sum(fsp.total_revenue)) DESC
      Sort Method: quicksort Memory: 168kB
      -> HashAggregate (cost=576.22..583.94 rows=772 width=77) (actual time=4.219..4.404 rows=917 loops=1)
        Group By Key: ds.first_name, ds.last_name, fsp.store_id, dd.date_id, fsp.num_rentals, fsp.total_revenue, fsp.staff_id
        -> Hash Join (cost=38.96..560.78 rows=772 width=45) (actual time=0.577..3.510 rows=917 loops=1)
          Hash Cond: (fsp.staff_id = ds.staff_id)
          -> Hash Join (cost=36.72..547.93 rows=772 width=32) (actual time=0.434..3.112 rows=917 loops=1)
            Hash Cond: (fsp.date_id = dd.date_id)
            -> Row Adapter (cost=445.40..445.40 rows=15648 width=31) (actual time=0.127..1.181 rows=15656 loops=1)
              -> CStore Scan on fact_staff_performance fsp (cost=0.00..445.40 rows=15648 width=31) (actual time=0.118..0.523 rows=15656 loops=1)
                Filter: (store_id = 2)
                Rows Removed by Filter: 15601
              -> Hash (cost=34.29..34.29 rows=194 width=9) (actual time=0.193..0.193 rows=194 loops=1)
                Buckets: 32768 Batches: 1 Memory Usage: 264kB
                -> Index Scan using idx_is_holiday on dim_date dd (cost=0.00..34.29 rows=194 width=9) (actual time=0.095..0.167 rows=194 loops=1)
                  Index Cond: (is_holiday = true)
                  Filter: is_holiday
            -> Hash (cost=1.55..1.55 rows=55 width=17) (actual time=0.040..0.040 rows=55 loops=1)
              Buckets: 32768 Batches: 1 Memory Usage: 259kB
              -> Seq Scan on dim_staff ds (cost=0.00..1.55 rows=55 width=17) (actual time=0.018..0.026 rows=55 loops=1)
          -> Seq Scan on dim_staff ds (cost=0.00..1.55 rows=55 width=17) (actual time=0.018..0.026 rows=55 loops=1)
        -> Seq Scan on dim_staff ds (cost=0.00..1.55 rows=55 width=17) (actual time=0.018..0.026 rows=55 loops=1)
      -> Seq Scan on dim_staff ds (cost=0.00..1.55 rows=55 width=17) (actual time=0.018..0.026 rows=55 loops=1)
    -> Seq Scan on dim_staff ds (cost=0.00..1.55 rows=55 width=17) (actual time=0.018..0.026 rows=55 loops=1)
  -> Seq Scan on dim_staff ds (cost=0.00..1.55 rows=55 width=17) (actual time=0.018..0.026 rows=55 loops=1)
Total runtime: 23.211 ms
26 rows)

```

Table 4: Query Execution Time Comparison Before Indexing Critical Columns and After Indexing Critical Columnna

Before/After Indexing	Query Time (ms)
Before Indexing Critical Columns	35.184
After Indexing Critical Columns	23.211

5.2 Implement partitioning to improve query performance.

```

CREATE TABLE fact_staff_performance_partitioned (
  staff_id INT NOT NULL,
  store_id INT NOT NULL,
  date_id DATE NOT NULL,
  num_rentals INT,
  num_unique_customers INT,
  total_revenue DECIMAL(10, 2),
  avg_transaction_value DECIMAL(10, 2),
  CONSTRAINT pk_fact_staff_performance_partitioned PRIMARY KEY (staff_id,
store_id, date_id)
)
PARTITION BY RANGE (date_id) (
  PARTITION p_2012_12 VALUES LESS THAN ('2012-12-31'),
  PARTITION p_2014_12 VALUES LESS THAN ('2014-12-31'),
  PARTITION p_2016_12 VALUES LESS THAN ('2016-12-31'),
  PARTITION p_2018_12 VALUES LESS THAN ('2018-12-31'),
  PARTITION p_2020_12 VALUES LESS THAN ('2020-12-31')
);

INSERT INTO fact_staff_performance_partitioned

```

```
SELECT * FROM fact_staff_performance;
```

```
EXPLAIN ANALYZE
```

```
SELECT
```

```
    staff_id,  
    store_id,  
    date_id,  
    num_rentals,  
    num_unique_customers,  
    total_revenue,  
    avg_transaction_value
```

```
FROM fact_staff_performance
```

```
WHERE date_id BETWEEN '2012-01-01' AND '2020-12-31'
```

```
ORDER BY store_id, date_id;
```

```
EXPLAIN ANALYZE
```

```
SELECT
```

```
    staff_id,  
    store_id,  
    date_id,  
    num_rentals,  
    num_unique_customers,  
    total_revenue,  
    avg_transaction_value
```

```
FROM fact_staff_performance_partitioned
```

```
WHERE date_id BETWEEN '2012-01-01' AND '2020-12-31'
```

```
ORDER BY store_id, date_id;
```

Output (Before):

```
dvd=# EXPLAIN ANALYZE
SELECT
    staff_id,
    store_id,
    date_id,
    num_rentals,
    num_unique_customers,
    total_revenue,
    avg_transaction_value
FROM fact_staff_performance
WHERE date_id BETWEEN '2012-01-01' AND '2020-12-31'
ORDER BY store_id, date_id;

                                QUERY PLAN
-----
Row Adapter  (cost=2618.36..2618.36 rows=27480 width=46) (actual time=37.706..43.105 rows=27471 loops=1)
-> Vector Sort  (cost=2549.66..2618.36 rows=27480 width=46) (actual time=37.667..40.246 rows=27471 loops=1)
    Sort Key: store_id, date_id
    Sort Method: quicksort  Memory: 6945kB
    -> CStore Scan on fact_staff_performance  (cost=0.00..523.54 rows=27480 width=46) (actual time=0.089..1.061 rows=27471 loops=1)
        Filter: ((date_id >= '2012-01-01 00:00:00'::timestamp without time zone) AND (date_id <= '2020-12-31 00:00:00'::timestamp without time zone))
        Rows Removed by Filter: 3786
Total runtime: 64.542 ms
(8 rows)
```

Output (after):


```

SELECT
  staff_id,
  store_id,
  date_id,
  num_rentals,
  num_unique_customers,
  total_revenue,
  avg_transaction_value
FROM fact_staff_performance_partitioned
WHERE date_id BETWEEN '2012-01-01' AND '2020-12-31'
ORDER BY store_id, date_id;

-----
QUERY PLAN
-----
Sort (cost=717.84..718.19 rows=140 width=56) (actual time=50.316..52.618 rows=27471 loops=1)
  Sort Key: store_id, date_id
  Sort Method: quicksort  Memory: 3344kB
  -> Partition Iterator (cost=0.00..712.85 rows=140 width=56) (actual time=0.033..19.635 rows=27471 loops=1)
    Iterations: 5
    -> Partitioned Seq Scan on fact_staff_performance_partitioned (cost=0.00..712.85 rows=140 width=56) (actual time=0.072..16.663 rows=27471 loops=5)
      Filter: ((date_id >= '2012-01-01 00:00:00'::timestamp without time zone) AND (date_id <= '2020-12-31 00:00:00'::timestamp without time zone))
      Rows Removed by Filter: 3786
      Selected Partitions: 1..5
Total runtime: 55.752 ms
(10 rows)

```

Table 3: Query Execution Time Comparison For Non-Partitioned vs Partitioned Tables

Tables	Query Time (ms)
Non-partitioned (fact_staff_performance)	64.542
Partitioned (fact_staff_performance_partitioned)	55.752

5. Conclusion

To conclude, we created a dimensional database from the Sakila database, focusing on staff performance metrics. This database allows for detailed analysis of staff activities, including rentals, unique customers served, and revenue generated. However, the process was not without challenges. Initially, we selected the MBTA database from the MIT website, but its unconventional structure and lack of alignment with typical business operations made it unsuitable for building a dimensional model, leading to a last-minute change. Switching to the Sakila database provided a better fit, but we encountered missing records that required importing the complete dataset. Despite these obstacles, we overcame them to deliver a functional and insightful database model.

6. **Appendix A: Instructions on Importing Missing Records into Database**

Step 1: Follow Set up Instructions provided on Ebwise

Step 2: Connect to dvd database which is already included in the opengaussDB after setting up.

To check the list of databases in openGauss:

\\

Make sure dvd exists as shown below:

```
openGauss=# \l
```

List of databases					
Name	Owner	Encoding	Collate	Ctype	Access privileges
d1	omm	UTF8	C	C	
d10	omm	UTF8	C	C	
d11	omm	UTF8	C	C	
d12	omm	UTF8	C	C	
d2	omm	UTF8	C	C	
d3	omm	UTF8	C	C	
d4	omm	UTF8	C	C	
d5	omm	UTF8	C	C	
d6	omm	UTF8	C	C	
d7	omm	UTF8	C	C	
d8	omm	UTF8	C	C	
d9	omm	UTF8	C	C	
dvd	omm	UTF8	C	C	
l1	omm	UTF8	C	C	
mbta	omm	UTF8	C	C	
nyc	omm	UTF8	C	C	
postgres	omm	UTF8	C	C	
t1	ogc	UTF8	C	C	
template0	omm	UTF8	C	C	=c/omm omm=CTc/omm
template1	omm	UTF8	C	C	=c/omm omm=CTc/omm

(20 rows)

Then check the number of tables and records:

```
SELECT 'actor' AS table_name, COUNT(*) FROM public.actor
UNION ALL
SELECT 'address' AS table_name, COUNT(*) FROM public.address
UNION ALL
SELECT 'category' AS table_name, COUNT(*) FROM public.category
UNION ALL
SELECT 'city' AS table_name, COUNT(*) FROM public.city
UNION ALL
SELECT 'country' AS table_name, COUNT(*) FROM public.country
UNION ALL
SELECT 'customer' AS table_name, COUNT(*) FROM public.customer
```

```
UNION ALL
SELECT 'film' AS table_name, COUNT(*) FROM public.film
UNION ALL
SELECT 'film_actor' AS table_name, COUNT(*) FROM public.film_actor
UNION ALL
SELECT 'film_category' AS table_name, COUNT(*) FROM
public.film_category
UNION ALL
SELECT 'inventory' AS table_name, COUNT(*) FROM public.inventory
UNION ALL
SELECT 'language' AS table_name, COUNT(*) FROM public.language
UNION ALL
SELECT 'payment' AS table_name, COUNT(*) FROM public.payment
UNION ALL
SELECT 'rental' AS table_name, COUNT(*) FROM public.rental
UNION ALL
SELECT 'staff' AS table_name, COUNT(*) FROM public.staff
UNION ALL
SELECT 'store' AS table_name, COUNT(*) FROM public.store
UNION ALL
```

Output:

table_name	count
actor	200
address	0
category	16
city	600
country	109
customer	599
film	1000
film_actor	5462
film_category	1000
inventory	4581
language	6
payment	14596
rental	16044
staff	2
store	2
zfact_sales	14596

(16 rows)

- There should be 15 tables in the sakila database. Ignore zfact_sales. Some of the tables are missing records, so check this [github repository](#) and then import the csv files accordingly.

Step 3: IMPORTING MISSING RECORDS INTO SAKILA DATABASE

1. Download and Prepare the sakila_missing folder and store it in your Windows C drive.
2. Files in the sakila_missing Folder:

The **sakila_missing** folder includes a subfolder and CSV files for each of these table in the Sakila database:

- a. address
- b. customer
- c. payment
- d. rental
- e. staff
- f. store

3. Copy Files to Linux VM:

- a. Open a **command prompt** on your Windows system.

- b. Run the following command to copy the **sakila_missing** folder to your **Linux VM**:

```
scp -r C:\sakila_missing root@192.168.0.40:/home/omm/
```

This will copy the **sakila_missing** folder (which contains the CSV files) to the **/home/omm/** directory on your Linux VM

4. Log in to the Linux VM:

- a. Start your **Linux VM** and log in using your **hjr** credentials (username: root, password: 1).
- b. Switch to the **omm** user by typing:

```
su - omm
```

This ensures you have the correct permissions for managing the database.

c.

5. Start the Database:

- a. To start the **GaussDB** instance, run:

```
gs_om -t start
```

This brings the database online.

6. Connect to the Database:

- a. Open an SQL session with superuser privileges by running:

```
gsq1 -d postgres -r
```

7. Verify Tables:

- a. Check if the Sakila database is created and the tables exist by running:

```
\c dvd  
\dt
```

8. Clear Existing Data in Tables:

- a. Run the following commands to **truncate** (clear) the existing data in the tables that are missing records:

```
TRUNCATE TABLE address, customer, payment, rental, staff,  
store CASCADE;
```

9. Import Data from CSV Files:

- a. Import the missing records from the **missing_records** folder into the corresponding tables using the **COPY** command. Run these commands one by one:

```
COPY address FROM '/home/omm/missing_records/address.csv'  
DELIMITER ',' CSV;
```

```
COPY customer FROM  
'/home/omm/missing_records/customer.csv' DELIMITER ','  
CSV;  
COPY payment FROM '/home/omm/missing_records/payment.csv'  
DELIMITER ',' CSV;  
COPY rental FROM '/home/omm/missing_records/rental.csv'  
DELIMITER ',' CSV;  
COPY staff FROM '/home/omm/missing_records/staff.csv'  
DELIMITER ',' CSV;  
COPY store FROM '/home/omm/missing_records/store.csv'  
DELIMITER ',' CSV;
```